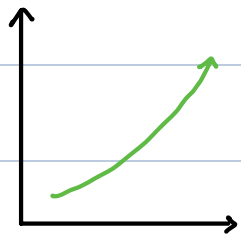
Agenda:

1. Painter's Partition - Find min time to paint
2. Aggressive cows - Find max count of cows
3. Aggressive cows - Find max possible min distance

$f(x) = x^2 + 5x + 3$

50

$f(x) \propto x$



Painter's Partition Problem

Given N boards with length of each board

a) A painter takes 1 unit of time to paint 1 unit of length.

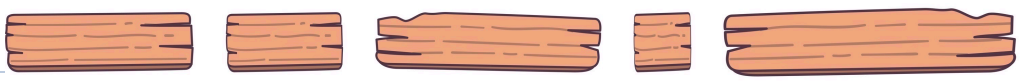
b) A board can only be painted by 1 painter.

c) A painter can only paint boards placed next to each other (i.e continuous segment).

Q: Find the **minimum time** required to paint all the boards if **P** painters are available**.**

N = **5**

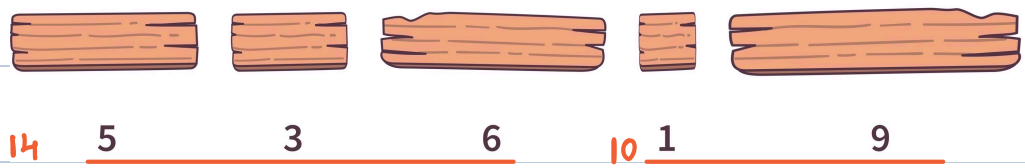
P = **3**



8 5 3 7 6 1 9 9

time = max (8, 7, 9) = 9

P = 2



time = max (14, 10) = 14

P	time
1	24
2	14
3	9

P ∝ 1/time

Ans → time

1) $l = \forall i, \max A[i]$ ← O(N) $r = \sum_{\forall i} A[i]$ // P = 1 ← O(N)

```
while (l <= r) {
    m = l + (r - l)/2
2)  // check if min time to paint all boards
    with P painters = m.
    cnt = countPainters (A, m)      // last class   ← O(N)
    cnt1 = countPainters (A, m-1)   // m is minimum
    if (cnt <= P && cnt1 > P)   return m
3)  if (cnt > P)    l = m + 1
    else            r = m - 1
}
```

TC = $O(N * \log(\sum A[i] - \max A[i]))$

```
int countPainters (A[], T) {                    SC = O(1)

    ans = 1      time = T

    for i → 0 to (N-1) {

        if (A[i] > T)   return ∞   // not possible

        if (time >= A[i])     time = time - A[i]

        else {   ans++

                 time = T - A[i]
            }
    }  return ans                    TC = O(N)      SC = O(1)
}
```

Aggressive Cows

A farmer has build a barn with **M** stalls. All **M** stalls are located at the different locations at x-axis, place all the cows such that **minimum distance** between any two cows is **maximised**.

A[i] = Location of the i-th stall in **sorted** order.

Q1: Find the **maximum number of cows** that can be placed if all pair of cows should at-least be **D** distance apart.

C_1 C_2 C_3

A = [1, 3, 6, 8, 10] D = 4 Ans = 3

C_1 C_2 C_3 C_4

A = [2, 6, 11, 14, 19, 25, 30, 39, 43] D = 10 Ans = 4

C_4 C_3 C_2 C_1

```
cnt = 1        last = A[0]        // place first cow at A[0]
for i -> 1 to (N-1) {
    if (A[i] - last >= D) {
        cnt++        last = A[i]
    }
}
return cnt
```

TC = O(N) SC = O(1)

Q2: Find **maximum possible minimum distance** between any pair of cows if farmer has **C** cows.

$c_1 \xleftrightarrow{5} c_2 \xleftrightarrow{4} c_3$ ✓

A = [1, 3, 6, 8, 10] C = 3

$c_1 \xleftrightarrow{5} c_2 \xleftrightarrow{2} c_3$ ✗

$c_1 \xleftrightarrow{9} c_2 \xleftrightarrow{8} c_3 \xleftrightarrow{11} c_4$ ✗

A = [2, 6, 11, 14, 19, 25, 30, 39, 43] C = 4

$c_1 \xleftrightarrow{12} c_2 \xleftrightarrow{11} c_3 \xleftrightarrow{14} c_4$ ✗

$c_1 \xleftrightarrow{12} c_2 \xleftrightarrow{16} c_3 \xleftrightarrow{13} c_4$ ✓

D ↑↓ C ↓↑

$D \propto 1/C$ ↗ #cows

↓

min distance b/w any 2 pair of cows

Ans → min distance

1) $l = 0$ $r = A[N-1] - A[0]$ // $c = 0$

```
while (l <= r) {
    m = l + (r-l)/2
2)  // check if m is max possible min distance
    b/w any 2 consecutive cows if #cows = C
    cnt = countCows(A, m)
    cnt1 = countCows(A, m+1)  ← TC = O(N)
    if (cnt >= C && cnt1 < C)   return m
3)  if (cnt < C)   r = m-1   // not able to place C cows
    else   l = m+1
}
```

TC = O(N log (Range of A[i])) SC = O(1)

organised data / conditions → 2 parameters → $p1 \propto p2$ OR $p1 \propto 1/p2$

i/p → min / max to find

What will be the output for following code snippet?

```
public class Test {
    static boolean canPaint(int[] arr, int k, int limit) {
        int painters = 1, sum = 0;          2        45
        for (int x : arr) {
            if (x > limit) return false;  x
            if (sum + x <= limit) sum += x;  15 18 45
            else {
                painters++;  // 2
                sum = x;     // 20
            }
        }
        return painters <= k;
    }

    public static void main(String[] args) {
        int[] boards = {5, 10, 30, 20};
        int k = 2;

        int lo = 30, hi = 65;
        int ans = -1;
```

```java
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;

            if (canPaint(boards, k, mid)) {
                ans = mid;
                hi = mid - 1;
            } else {
                lo = mid + 1;
            }
        }

        System.out.println(ans);
    }
}
```

$\frac{45+46}{2} = 45$

→ ~~47~~ 45

→ ~~46~~ 44

→ ~~39~~ ~~43~~ 45

→ 45

For the following Code Snippet , What is the correct fix?

```java
static boolean isPossible(int[] stalls, int dist, int cows) {
    int placed = 1;
    int last = stalls[0];

    for (int i = 1; i < stalls.length; i++) {
        if (stalls[i] - last >=dist) {  // Bug here
            placed++;
            last = stalls[i];
        }
        if (placed == cows) return true;
    }
    return false;
}
```

What will be the output for following code snippet?

```java
public class Test {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30};
        int lo = 30, hi = 60;
        int ans = -1;

        while (lo < hi) {
            int mid = (lo + hi) / 2;

            if (mid >= 30) {
                ans = mid;
                hi = mid;
            } else {
                lo = mid + 1;
            }
        }

        System.out.println(ans);
    }
}
```

// $\frac{30+31}{2} = 30$

→ ~~45~~ ~~37~~ ~~33~~ ~~31~~ 30

→ ~~45~~ ~~37~~ ~~33~~ ~~31~~ 30

→ 30